

Tập 2 - Pod Network

Giải phẫu Pause Container, veth pair & Network Namespace

Phần 0 — Nền tảng K8s Networking · #pause #veth #namespace #linux



Lộ trình bài học: Lab First - Slide Second

Hôm nay chúng ta sẽ tiếp tục bóc tách "thế giới ngầm" của Pod bằng cách làm trước, lý giải sau:

- **PHẦN 1: Thực hành điều tra (Lab First)**

- Lên vào Network Namespace của Pod từ OS vật lý bằng `nsenter`.
- Lùng sục "sợi dây cáp ảo" `veth` và switch ảo `cni0`.
- Giả lập crash ứng dụng để xem mạng K8s bền vững thế nào.

- **PHẦN 2: Giải phẫu kiến trúc (Slide Second)**

- Sơ đồ hóa Pod thực chất là gì dưới góc nhìn Linux Kernel.
- Giải nghĩa vai trò của "kẻ canh giữ mạng" — Pause Container.
- Làm rõ nguyên lý sợi dây cáp ảo `veth pair`.

PHẦN 1: Bắt đầu làm Lab ngay!

Chúng ta sẽ thực hiện các Thí nghiệm đột phá trong file `lab-guide.md`:

1. **Thí nghiệm 1 & 2:** Tìm PID của pause container và dùng `nsenter -t <PID> -n ip addr` để nhìn trộm card mạng của Pod từ máy Host.
2. **Thí nghiệm 3:** Truy tìm sợi cáp mạng ảo `veth` và xem cách nó cắm vào bridge `cni0`.
3. **Thí nghiệm 4:** Dùng `crictl stop` để kill chết container ứng dụng, chứng kiến container hồi sinh nhưng IP mạng vẫn giữ nguyên tuyệt đối.

👉 Hãy tạm dừng video, mở terminal và bắt đầu gõ lệnh trong tệp `lab-guide.md` nhé!

PHẦN 2: Đúc kết từ Lab — Hiện tượng kỳ bí

Sau bài Lab vừa rồi, chúng ta gặp 2 hiện tượng cực kỳ thú vị:

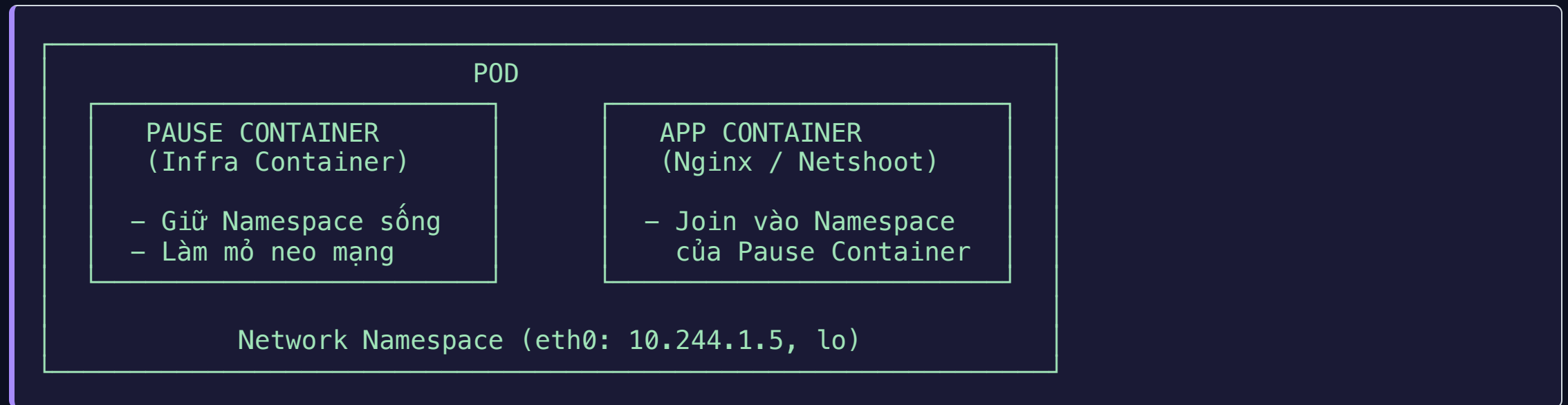
1. **Hiện tượng 1:** Địa chỉ IP mạng của Pod thực chất lại hiển thị dưới PID của một tiến trình có tên là `/pause` chứ không phải PID của container ứng dụng (Nginx/Netshoot).
2. **Hiện tượng 2:** Khi chúng ta cố tình "hạ sát" container ứng dụng bằng lệnh `crictl stop`, Kubernetes tự khởi động lại container mới, cột RESTARTS tăng lên 1, nhưng địa chỉ IP của Pod hoàn toàn **không bị thay đổi!**

👉 *Tại sao lại như vậy? Chúng ta hãy cùng bóc tách lý thuyết.*

Sơ đồ "Thế giới ngầm" của một Pod

Hầu hết các tài liệu cơ bản nói: "*Pod là container*". Điều này **chưa chính xác**.

Thực chất, **Pod = Nhóm các container chia sẻ cùng một Network Namespace**:

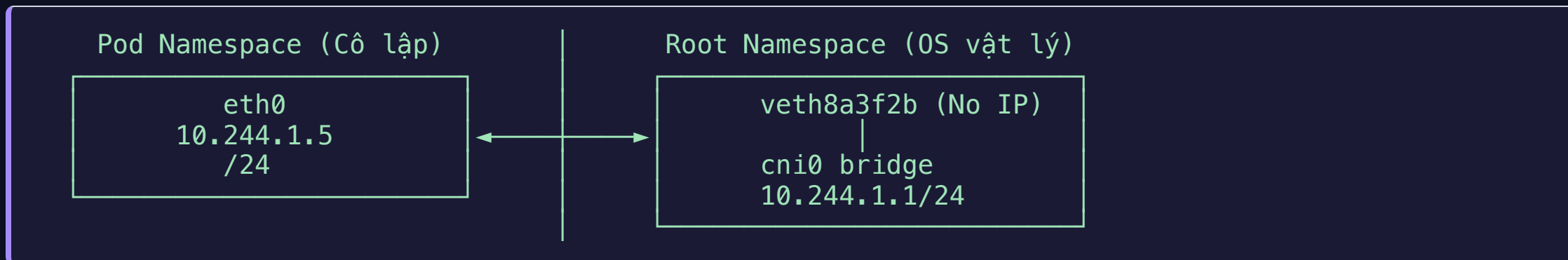


- **Pause container** được sinh ra đầu tiên để tạo ra Network Namespace và "neo" nó lại.
- Nếu App container bị crash và biến mất, Network Namespace vẫn tồn tại nhờ Pause container vẫn sống. Khi App container khởi động lại, nó chỉ cần join lại namespace đó → **IP Pod được giữ vững!**

veth pair: Sợi cáp ảo kết nối Pod ra ngoài Host

Làm thế nào gói tin đi ra khỏi Network Namespace bị cô lập của Pod để lên OS vật lý?

CNI sử dụng cơ chế **veth pair** (**Virtual Ethernet Pair**) giống như một sợi dây cáp ảo:



- **veth pair** luôn được tạo thành cặp: 1 đầu cắm vào Pod (đặt tên là `eth0`), 1 đầu nằm ngoài host (đặt tên ngẫu nhiên như `vethXXXX`).
- Mọi gói tin chui vào đầu `eth0` trong Pod sẽ tự động chọt ra đầu `vethXXXX` ngoài host, và được chuyển tiếp trực tiếp vào switch ảo `cni0`.

Key Takeaways — Bài học cốt lõi

- **Pause container là anh hùng thầm lặng:** Nhẹ (~300KB), không làm gì ngoài việc ngủ (`sleep infinity`), nhưng chịu trách nhiệm giữ sinh mệnh mạng cho Pod.
- **veth pair và bridge:** Cấu trúc mạng K8s cục bộ hoạt động y hệt cách bạn cắm dây mạng LAN từ máy tính vào một Switch vật lý trong nhà.
- **nsenter** là vũ khí debug tối thượng: Giúp kỹ sư hệ thống bỏ qua lớp bảo mật K8s để trực tiếp nhảy vào Linux Kernel kiểm tra cấu hình mạng.

Tập tiếp theo: IP của Pod rất dễ thay đổi khi Pod bị xóa hẳn. Làm thế nào để tạo một địa chỉ IP ảo cố định?
Cùng tìm hiểu Service & kube-proxy!